DATE ___/___/___

Circuit Simulation

- A circuit simulator is provided with:
  - → Circuit connectivity by means of a netlist.
  - → Component behaviour by means of device models and model parameters.
  - → The initial state of the circuit, known as initial conditions.
  - → Something that is input to the circuit, called stimulus.

- This information is used to:-
  - → Construct a set of ordinary differential equations that describe the circuit.
  - → Solve the equations using nodal analysis, which for a circuit containing resistors, capacitors and current sources is simply:

$$i(v(t)) + \frac{d}{dt} q(v(t)) + u(t) = 0$$

$$V(0) = c$$

- $i(v(t))$: current (i) entering nodes (v) from resistors
- $\frac{d}{dt} q(v(t))$: Charge (q) entering nodes (v) from capacitors
- $u(t)$: current sources

DATE ___/___/___

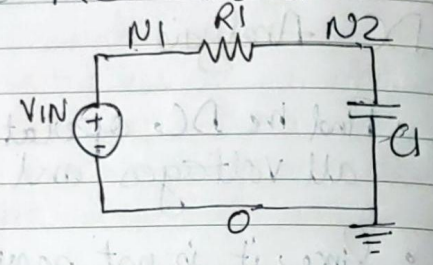
Main Analysis Types

- DC Analysis
  - Find the DC operating point of the circuit i.e. all voltages and currents.
  - Since it is not possible to explicitly solve a system of nonlinear algebraic equations, the systems are linearized and solved using Newton's method.
  - This is an iterative process we run to convergence.
- Transient Analysis
  - Since there is no known method to solve the nonlinear differential equations, the simulator discretizes time.
  - In other words, it solves a DC Analysis for each timestamp based on initial conditions.
- AC Analysis

Find the DC operating point and assume linearity for small signals

DATE __/__/____

Example: A Simple RC Circuit



RC Circuit

*** SETTINGS ***
simulator lang = spice

*** NETLIST ***
** Parameters:
.PARAM vdd=5 *Supply voltage

** Voltage Source **
*VIN N1 0 AC 5 SIN (0 5 1MEG)
VIN N1 0 PULSE(0 vdd 1u) * 5V pulse with 1us delay

** Elements **
R1 N1 N2 5
C1 N2 0 1u

*** Analysis ***
*.AC DEC 10 0.1 100MEG
*.PRINT AC V(N2)
.TRAN 1n 200u * 200 µs simulation with 1ns steps
.MEAS TRAN tau TRIG V(N2) VAL=0.000001 RISE=1 TARG V(N2) VAL='0.63*vdd' RISE=1
.PRINT TRAN V(N2)

.END

DATE __/__/____

Why DC Operating Point?

- Every simulation starts with calculation of DC Operating Point (DC OP).

⇒ .op calculates the voltage, current, power, etc at each component.

⇒ .dc computes the .op point as a function of some independent variable.

The Important

- .ac and .noise first compute the DC OP and then linearize the circuit around it to compute the small-signal behaviour of the circuit.
- tran computes the DC OP for an initial state of the circuit. Transient simulations are then basically a collection of sequential DC OPs starting with these conditions.

DC Analysis Starting Point

- DC OPs are equilibrium points, i.e. do not change with time,

DATE

- Independent sources are configured to be constant.
- $dv/dt = 0 \rightarrow$ Capacitors are open circuits.
- $di/dt = 0 \rightarrow$ Inductors are short circuits

- DC OP Starting State

→ All Caps and Inductors are removed.
→ DC Sources values are assigned.
→ Node Sets are assigned.
→ Time Varying part of signal source is ignored.
→ Initial Conditions are ignored.

Now Nodal Analysis can be applied.

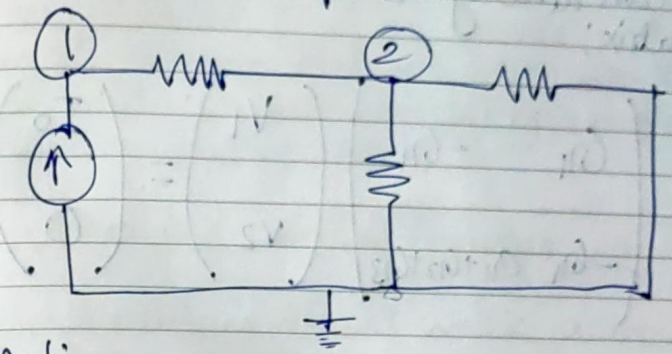
$$i(v(t)) + \frac{d}{dt} q(v(t)) + u(t) = 0$$
$$v(0) = a$$

$$\Rightarrow \frac{d}{dt} q(v(t)) = 0 \rightarrow i(V_{dc}) + U_{dc} = 0$$

DC OP Nodal Analysis

DATE

- We will write KCL using branch constitutive equations



Node 1:

$$\frac{V_2 - V_1}{R_1} + I_0 = 0$$

Node 2:

$$\frac{V_1 - V_2}{R_1} - \frac{V_2}{R_2} - \frac{V_2}{R_3} = 0$$

Let's now replace with $G_1 = \frac{1}{R}$ and we have:

$$G_1(V_2 - V_1) + I_0 = 0$$
$$\hookrightarrow G_1 V_1 - G_1 V_2 = I_0$$
$$G_1(V_1 - V_2) - G_2 V_2 - G_3 V_2 = 0$$
$$\hookrightarrow -G_1 V_1 + (G_1 + G_2 + G_3) V_2 = 0$$

$$\begin{pmatrix} G_1 & -G_1 \\ -G_1 & G_1 + G_2 + G_3 \end{pmatrix} \begin{pmatrix} V_1 \\ V_2 \end{pmatrix} = \begin{pmatrix} I_0 \\ 0 \end{pmatrix}$$

DATE / /

- We can directly write the conductance matrix:

$$\begin{pmatrix} G_1 & -G_1 \\ -G_1 & G_1+G_2+G_3 \end{pmatrix} \begin{pmatrix} V_1 \\ V_2 \end{pmatrix} = \begin{pmatrix} I_0 \\ 0 \end{pmatrix}$$

- Diagonal Elements:
  For Element a_{ii} add all conductances attached directly to node i.
- Off Diagonal Elements:
- For Element a_{ij}, add all conductances connected directly between node i and node j. Multiply by -1.
- For most off-diagonal elements there are no direct connections, so we get a sparse matrix which is easy to solve.

DATE / /

Source Vector (I):-

- For element i of the source vector, sum all source currents, flowing into node i and substract all currents flowing out of node i.

Modified Nodal Analysis

- Nodal analysis doesn't work with voltage sources.
- However we know the voltage on its node, so we get an equation such as $V_1 = V_{in}$
- But we don't have an ohmic relationship on the voltage source, so we add another unknown to the voltage node vector (V) i.e. I_1

Newton Raphson Method

- Modified nodal analysis works for solving circuits made of linear elements.

→ Resistors, Capacitors, Inductors, Current/Voltage Sources.

DATE

- However, not all devices are linear
  - Diodes
  - BJTs
  - MOSFETs
  - ...
- Nodal Analysis attempts to solve equations of the form $I(V_{dc}) + U_{dc} = 0$

→ However, when nonlinear elements are part of the circuit, these are nonlinear algebraic systems and so cannot be solved directly.

→ We need an algorithmic approach to find a solution

Solving Nonlinear Equations

- The Newton-Raphson method/algorithm (aka "Newton's Method"):
  - Makes an initial guess on the solution $v^{(0)}$
  - Linearizes the circuit around the guess

$$J(v^{(0)}) = \frac{d}{dv} f(v^{(0)})$$

- Solves the resulting system of linear equations $v^{(1)}$

DATE

- Re-linearizes the circuit around the new point

$$J(v^{(1)}) = \frac{d}{dv} f(v^{(1)})$$

- Repeats until the process converges
- Formally, Newton's method solves: By repeatedly solving $f(\hat{v}) = 0$.

$$\text{or} \quad J(v^{(k)})\left(v^{(k+1)} - v^{(k)}\right) = -f(v^{(k)})$$

where

$$v^{(k+1)} = v^{(k)} - J^{-1}(v^{(k)}) f(v^{(k)})$$

Step 0: - Initialize
set $k \leftarrow 0$
choose $v^{(0)}$

Step 1: - Linearize about $v^{(k)}$

$J(v) = \partial f(v^{(k)}) / \partial v$
where J_f is the Jacobian of f.

Step 2: - Solve the linearized system

$v^{(k+1)} \leftarrow v^{(k)} - J_f^{-1}(v^{(k)}) f(v^{(k)})$

Step 3: - Iterate
set $k \leftarrow k+1$
If not converged, go to step 1.

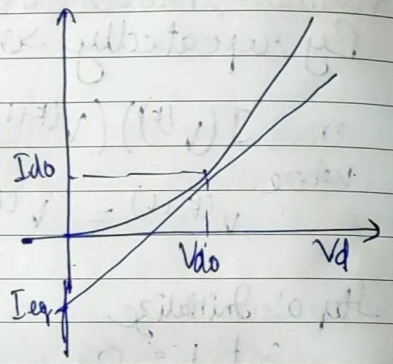
- First thing is the need to linearize our non-linear elements

DATE __/__/____

- For example, a diode.
- Linearizing the diode at the operating point gives us:

$$I_{DO} = G_{eq} \cdot V_{DO} + I_{eq}$$

- So, we replace the diode with this expression (equivalent to an I_{eq} current source in parallel to an R_{eq} resistor).



- But what is the operating point?

- It looks like a chicken and egg question.
- We need to know the operating point (I_{DO} and V_{DO}) to find G_{eq} and I_{eq}.
- But we are looking for the operating point aren't we?
- Exactly, so we'll guess, solve and check if we were right?

- If not, we'll use the solution as our guess, re-linearize, and solve again.

DATE __/__/____

Convergence

- When do we stop?
  We'll never get the exact answer. So we stop based on convergence criteria.

- The Residue Criterion
  - The first criterion is that KCL should be satisfied to a given degree $\left| f_n(V^{(k)}) \right| < \varepsilon_f$
- In practice, a relative criterion is used:

$$\sum I(\text{node } i) < \text{reltol} \cdot |I_{max}| + i_{abstol}$$

- reltol is by default 0.001
- i_{abstol} is by default 1pA

- The Update Criterion

The second criterion is that the difference in error between the last two iterations is small.

$$\left| V_n^{(k)} - V_n^{(k-1)} \right| < \varepsilon_x$$

- In practice, a relative criterion is used.
- v_{abstol} is by default 1μV

$$|V_n^{(k)} - V_n^{(k-1)}| < \text{reltol} \cdot \max(|V_n^{(k)}|, |V_n^{(k-1)}|) + \text{vabstol}$$

Changing Convergence Criteria

- What if the circuit fails to converge?
- Spectre performs maxiters iterations (default 150) only.
- This usually happens due to bad models or non-physical elements (non-continuities in the transfer curves).
- If convergence hasn't been met, it tries alternative methods to converge.
- These are homotopies
- There are 5 homotopy algorithms: gmin, source, dptran, ptran, none
- The default is 'all' which tries each algorithm one after the other.

- If you see that your solution is certain converging with a certain homotopy, you can select it without going through all of the options, and thus reduce runtime.

Convergence aids: minr

- Really small resistors cause terrible convergence problems.

→ A 1μV drop over a 1mΩ resistor results in 1KA of current.

→ This is easily due to a 'wrong guess' in the convergence process.

- The minr parameter:
  - All resistors with R < minr will be converted to minr.
  - The default is 1mΩ and shouldn't be reduced.
- In addition:
  - You shouldn't really extract small resistors when extracting parasitics.

- If you do then increase vabstol...

DATE

Convergence aids: gmin

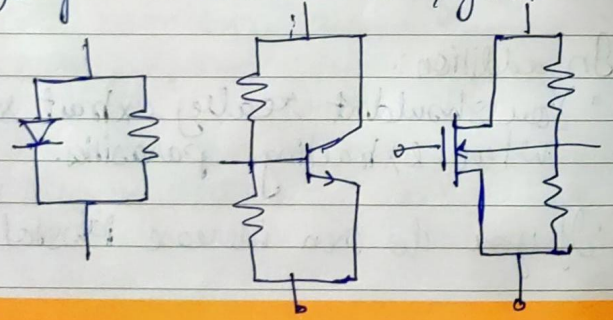
- Circuits with a non-isolated solution have a singular jacobian.

→ For example, if there is a floating component, there are infinite solutions.

→ Another example is a CMOS inverter with $V_{IN} = VDD/2$. If the FETs have infinite input impedance in saturation, the output has a range of voltages that satisfy KCL.

- Therefore, spectre automatically adds big resistors to ground between potentially floating nets.

→ This is done for all nonlinear components.
→ The size of the resistors is 1/gmin.



- For MOSFETs, a resistor is added between the source and drain!

The danger of gmin

- The gmin resistors are very big, so they will not affect the calculation.
  - By default, $gmin = 10^{12}$ ($R = 1T\Omega$)
- However, that may not always scale.
  - For example, for a 1V supply, this results in a 'fake' current of 1pA through a closed transistor.
  - Therefore, when dealing with small currents, gmin should be reduced substantially.

Convergence aids: nodeset

- Providing Newton's method with an initial guess is beneficial.

→ If the guess is close enough to the real DC point, convergence will be faster and will most likely succeed.

DATE

- To bias a multi-stable circuit (e.g. latch) towards a particular solution, an initial guess is provided with the nodeset option.
- Node Sets connect a DC source with a 1Ω resistor to a defined node.
- Node Sets are only used during the first convergence iteration and then released.
- Continuation methods (i.e. DC sweep) use the previous solution as the initial guess for the next simulation.
- The OP of the circuit can be saved to a file (write command) to load as a nodeset for a future simulation. (with the readns command)
- Add nodeset statements to q and qb to bias to one solution.

Saving and Loading Node Sets

- Two options for writting a nodeset file:

→ Write :- File will include state after initial DC solution.

→ Write final :- Will include state after final DC sweep.

- To load a saved nodeset as the initial guess of your simulation, use the readns option.
- Note that readforce and force are slightly different (generally don't use them).
- Another option is restart:
  - restart=yes (default) means the DC OP is calculated from the beginning if any change has occured.
- Usually, there's no reason to change this.

Start with a DC Operating Point

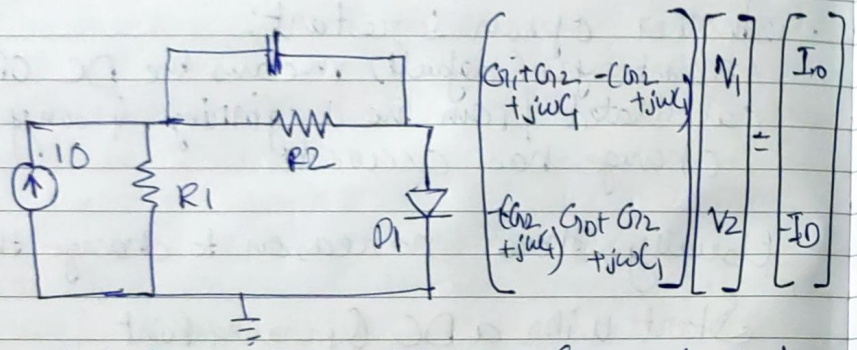
- An AC Analysis needs a bias point.
  - Equivalent to a DC operating point.
  - In other words, the bias point is found by DC OP convergence.
- However, due to phase shifts, the solution may be different.
- Usually due to setting a DC voltage on sources.

DATE

- One thing to do to eliminate some of these differences is

Do Not set the DC Voltage to 0 on Vpulse/VAC sources.

Comprise the AC Matrix



- Equivalent to the DC matrix, with Caps and inductors added as admittances.
  - jwc for Cap
  - 1/jwL for an inductor.

Transient Analysis

- Transient analysis generates a system of non linear ordinary differential equations.

DATE

- There is no known method to directly solve these equations.
- Instead discretize time:

e.g. using the Euler formulation

$$\frac{dq(t_i)}{dt} \approx \frac{q(t_i) - q(t_{i-1})}{t_i - t_{i-1}}$$

- This converts the problem into a system of non-linear algebraic equations.
- In other words:
  - first a DC OP is calculated (aka the "initial transient solution").
- Then caps and inductors are added and their currents and voltages are linearized according to their integral values.
- Non-linear elements are linearized.
- Newton-Raphson is used to find the DC OP for each time step throughout the transient.

DATE ___/___/___

- Timestamps are important to tradeoff accuracy vs runtime.

- After each timestep, the simulator:
  - Decides when the next timestamp should be.
  - Predicts the values at the next timestep.
  - Calculates the DC OP at the next timestep.
  - If the error between the prediction and calculation is bigger than the Local Truncation Error (LTE), a closer (sooner) timestamp is taken.
  - The ~~maximum~~ maxstep parameter sets the maximum allowed size of a timestamp.

- Breakpoints are set where pre-known discontinuities are simulated.
  - Such as at VPULSE or VPWL point.

- Calculations around breakpoints are handled separately to ensure convergence.

Local Truncation Error

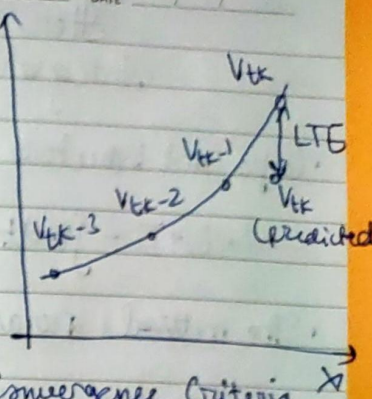
- LTE is the error between the calculated and predicted values at the next timestep.

DATE ___/___/___

- If the LTE is:

→ Smaller than the convergence criteria, the timestep is kept.

→ Otherwise, a closer timestep is chosen.



LTE: $|V_{tk} - V_{tk(predicted)}|$ < Convergence Criteria: $|Iteratio \cdot (reltol \cdot V_{tkcmax} + Vabstol)|$

- V_{tk}: Calculated value at current node
- $V_{tk(predicted)}$: Predicted value at current node
- reltol: Relative tolerance
- Vabstol: Absolute tolerance
- V_{tkcmax}: Maximum Voltage (relref)

relref sets the minimum voltage:
- pointlocal - the largest value at this node during this iteration.
- local - the largest value at this node so far.
- global - the largest value at any node so far.

DATE

Hot Integration Method

- Caps and inductors present an integral relationship
  - $L * I - \int V dt \qquad C * V = \int I dt$
- The method parameter sets the integration scheme
  - ⇒ euler (first order gear): only good around discontinuities $\frac{d}{dx} V(t_{k+1}) \approx \frac{1}{h}[V(t_{k+1}) - V(t_k)]$
  - → trap (trapezoidal): good but can cause oscillations $\frac{dv(k+1)}{dx} \approx \frac{2}{h}[V(t_{k+1}) - V(t_k)] - \frac{d}{dx} V(t_k)$
  - ⇒ gear2 (second order gear): good curve fitting but can damp oscillations $\frac{d}{dx} V(t_{k+1}) \approx \frac{3}{2h} V(t_{k+1}) - \frac{2}{h} V(t_k) + \frac{1}{2h} V(t_{k-1})$

DATE

- If you have "fake" oscillations, it's due to using trapezoidal integration.

errpreset

- Most of the important parameters for transient analysis are present according to the errpreset parameter:
  - liberal - fast simulations but can lose accuracy.
  - moderate - moderate simulation and accuracy.
  - conservative - high accuracy but slow.

errpreset	maxstep (maximum finest step)	reltol (Relative error Tolerance)	iterlte (LTE normalization)	relref	method (Maximum Integration method)
liberal	$\frac{Tstop - Tstart}{50}$	10	3.5	sigglobal	gear2
Moderate	$\frac{Tstop - Tstart}{50}$	1	3.5	sigglobal	trapgear2
conservative	$\frac{Tstop - Tstart}{100}$	0.1	10	alllocal	gear2only

Convergence Aids: Initial Conditions

DATE

- Initial Conditions are the same as Node Sets, but they are not removed after the first DC OP iteration.

→ Node sets are to assist in convergence.
→ Initial Conditions are to set a values at a node at the beginning of the transient.
→ DC OP and DC sweep analyses ignore initial conditions.

~~Convergence A~~

- Initial Conditions can be set on the test (.ic) or on caps/inds.

Saving Time Points

- Initial Conditions can also be written to a file and loaded to a simulation;

- write: file that ic of current simulation are written to.

- writefinal: file that final state of simulation is written to.

- read ic: - file to read initial condition form

DATE

- Other saving options:

- savedcdt: periodically save in real time minutes
- saveperiod: periodically save in simulation time.
- save time: save at specific simulation timepoints.
- save file: name of the save file.
- recover: start simulation from this file.
- inptimes: times to save DC OP.
- actimes: times to run AC simulation.

Other Stuff.

Tools and Version

The Cadence Custom IC Design includes the following tool suites:

- Virtuoso (IC 6.1.8): including schematic and layout editors, Analog Design Environment (ADE) VIVA etc.

- Spectre (NMSIM) including APS, Spectre X, XPS, Spectre AMS Designer, Spectre FX, Spectre X-RF etc.

DATE __/__/____

Different Simulation Options are:-

- Spectre SPICE engine.
- Spectre Accelerated Parallel Simulator (APS) for high-precision and scalable multicore simulation.
- Spectre X for high-performance, high-capacity simulation.
- Spectre Extensive Partitioning Simulator (XPS) an advanced fastSPICE engine.
- Spectre AMS Designer for mixing these with Xcelium.

Spectre APS

- Full baseline Spectre Accuracy.
  - No change to core Spectre timestamp algorithms.
- Same use model as baseline Spectre technology
  - Just add aps to the command line, or enable the ADE option.
- Significant performance gain on single/multiple core
  - 5-100x simulation performance vs baseline Spectre technology for non-RF.
  - 5-20x vs baseline Spectre RF
- Much larger simulation capacity
  - → Designs up to 10M transistors, 50M RCs (no reduction)
  - → EM/IR simulations with > 500M elements.
- Improved Convergence over core Spectre technology.

→ Unless your design is trivial in size, use the APS option.

Faster Spectre APS

- Designed to produce identical simulation results as baseline Spectre.
  - More accurate: Spectre + aps
  - Faster: spectre ++ aps
- Since ~~convergence~~ postlayout is often hard on convergence and transient simulation:
  - Spectre ++aps +postlayout
- Sometimes lightweight simulation can further speed things up
  - spectre +aps +lite

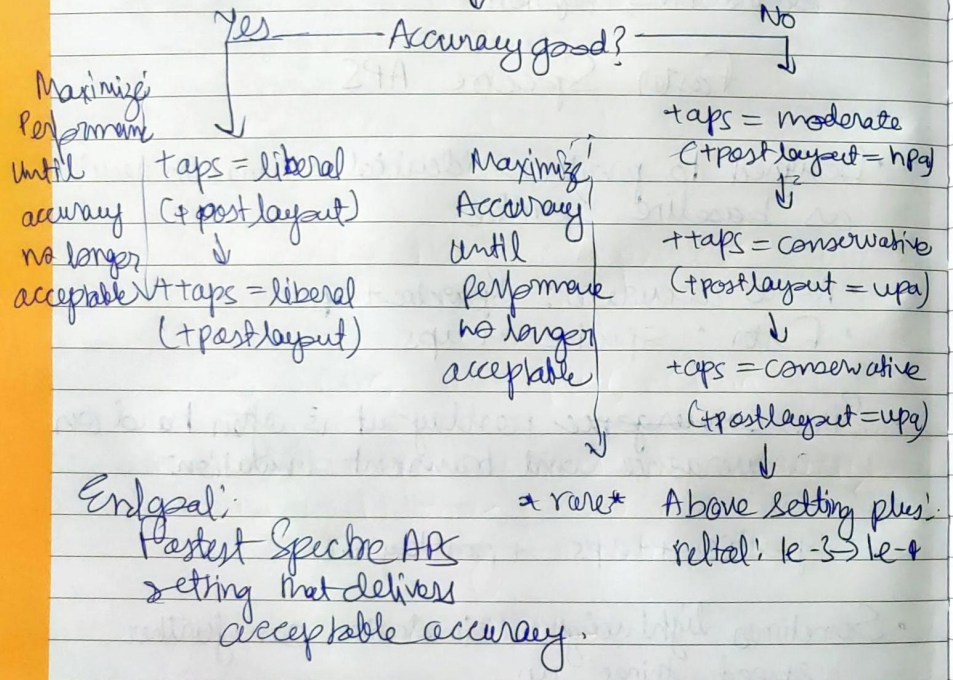
DATE __/__/____

DATE __/__/__

- To manually specify the number of multithreads (default = 8)
  - Spectre +aps +mt=16

Getting the Most Out of Spectre APS

++aps = moderate
(+postlayout = hpa)



End goal:
Fastest Spectre APS setting that delivers acceptable accuracy.

DATE __/__/__

Spectre X

Spectre X is the next generation of Spectre including:

- Enhanced simulation performance and capacity with a simple +preset use model.
- Highly scalable multi-core simulation with +mt, and distributed multi-process simulation with +xdp.
- To run Spectre X with a specific preset:
  → Spectre +preset = mx input.scs
  → These presets ignore errpreset options

Option Name	When to use...
cx	When a golden reference is needed
ax	for high precision analog applications
mx	for most analog applications (default)
lx	for power management and other relaxed analog applications
vx	for custom IC verification.

- For postlayout you can get:
  - Spectre +preset = mx +postlayout = cx

DATE __/__/__

- Spectre X uses the +xdp for distributed simulation on up to 512 cores.
- Check with your sysadmin what options can work.

Spectre XPS

- Spectre XPS is the new advanced FastSpice simulator.
- spectre +xps +mt=16 +ckt preset=dram +speed=1
- Spectre XPS has +ckt preset options for several circuit types.
  - dram, sram, sram-pwr, pcram, flash
- There also many options for reducing parasitics, partitioning, providing more accurate analog simulations etc.

Spectre MDL

DATE __/__/__

- Spectre MDL (Measurement Description Language) is a scripting language that enables you to define measurements and batch process simulations.
- Create an mdl control file
  - → 'export' defines measurements
  - → 'run' tells which analysis to run.
- Run the spectremdl command
  - spectremdl -batch myfile.mdl -design input.scs
- Postprocess the .raw data.
  - mdl -b test.mdl -r input.raw -m input.measure.